

Vending Machine Controller Project Documentation

1. Introduction *(Choi)*

In this report, we present the thorough development steps and the results of our hardware design and prototyping project. This report serves to describe how we managed to develop a fully functional **vending machine controller**. The main development steps are

1. Design in VHSIC Hardware Description Language (VHDL) on ModelSim.
2. Functional verification of the design using test benches on ModelSim.
3. FPGA design implementation on Vivado and hardware verification on the Nexys A7-100T development board.
4. Custom PCB development using KiCad.

The vending machine controller aims to have the following available features.

- Selection of items (using switches)
- Display of the names and the prices of available items (using 8 digits of 7-segment display)
- Insertion of coins (using push buttons)
- Display of the inserted coin amounts (using 8 digits of 7-segment display)
- Display of the current state (using LEDs)

The design implemented a Moore Finite State Machine to handle the user interactions as well as internal processes. In other words, our team used Moore FSM in VHDL design that allows the output to be determined by the current state only, which will be explained further in the following sections.

2. Team Members and Project Management *(Choi)*

2.1 Agile Development [2]

Our team followed the Agile Development approach as we learned it from Computer Science 2 class, previously at our university. Agile Development is an iterative and incremental development method that highlights the importance of flexibility, collaboration, and continuous improvement. Each team member had a leading role in each section of the project, but each member also gave feedback and helped one another in a weekly meeting (Scrum). In each meeting, we reviewed the current status of each task and shared feedback and challenges. In this way, we communicated very well within our group and took feedback from other members into the final result. Especially, agile method was especially well recognized for its effectiveness in hardware and software co-design projects and our vending machine controller project was thus successful.

2.2 Project Management using GitHub Repository and GitHub Project

To coordinate our project, we used GitHub Repository and GitHub project board. With these tools, we could organize different tasks successfully, specifically by following up each task progress and assigning responsibilities and deadlines for each task. With these tools, we could always be on the same page and be updated with the progress each member was making.

2.3 Roles and Responsibilities

Each member took a leading role in different parts of the project. Namely,

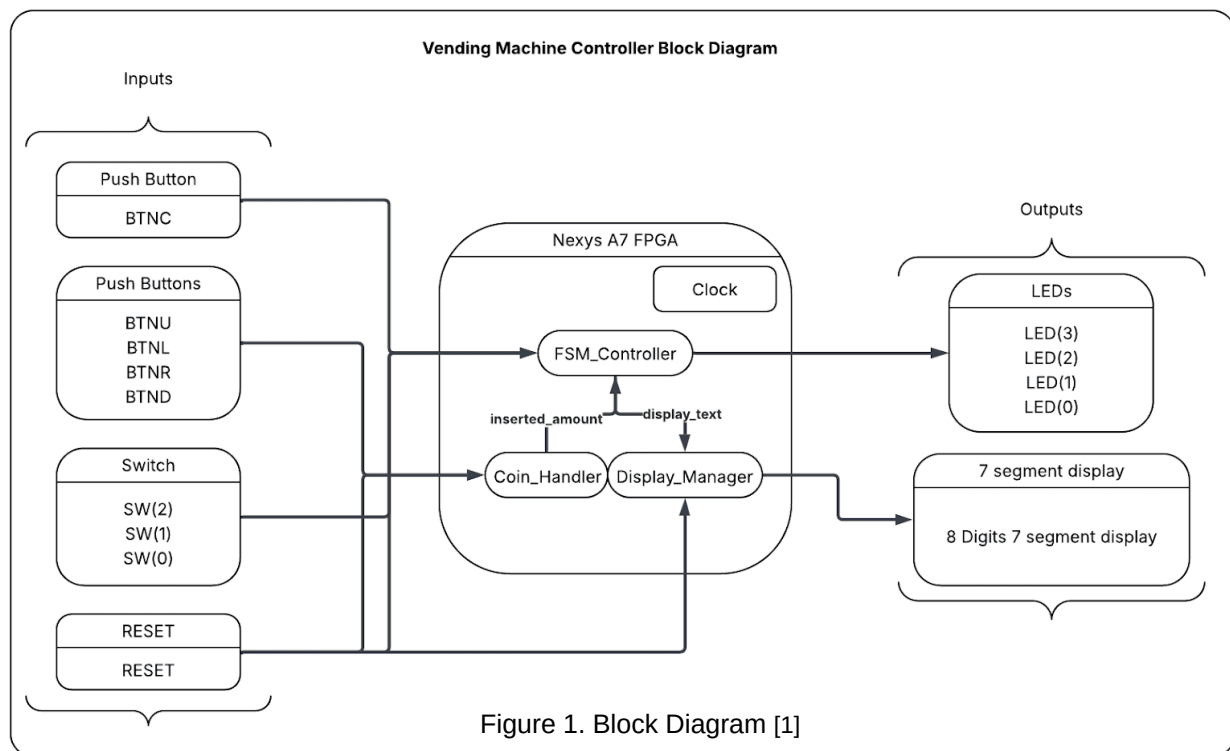
- Younsuk Choi: VHDL design of the FSM controller and peripheral integration, verification using test benches
- Yuming (Leano) Wang: VHDL code debugging and FPGA Implementation
- Muhammad Emaan Sajid: PCB Prototyping (Schematics, Layout, 3D view, BOM and Gerber Files)

As mentioned above, each member had a primary responsibility, while making sure that our collaboration was integral across all tasks. Each member not only led the development in each task but also helped other teammates in their respective tasks.

3. Concept Description and Requirements (*Choi*)

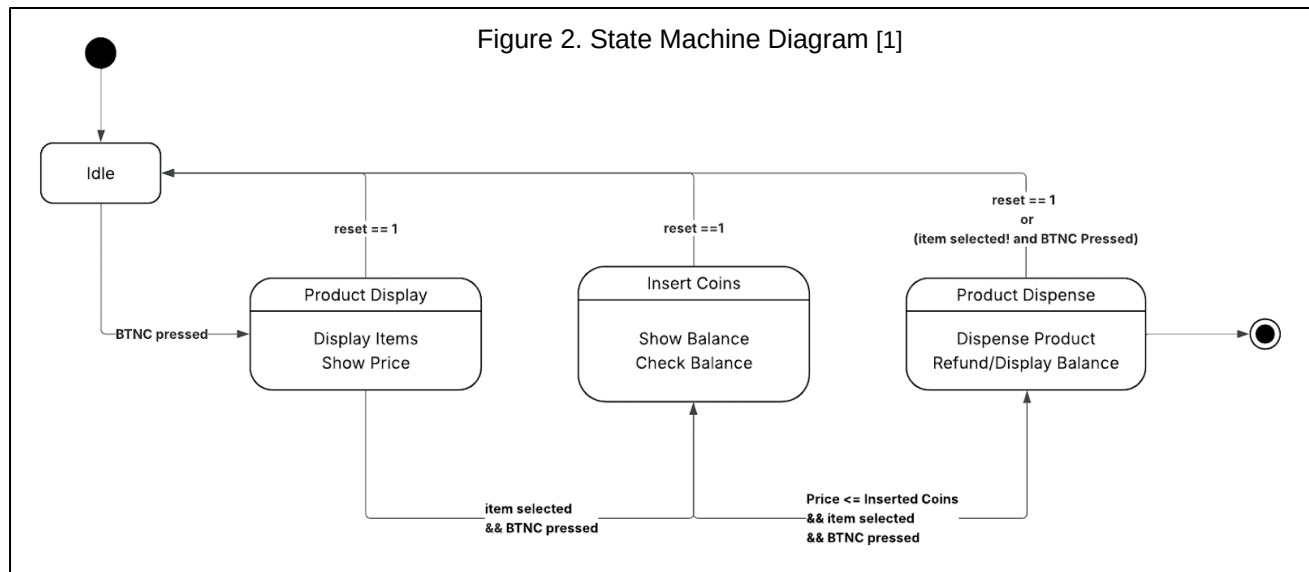
3.1 Vending Machine Controller Concept

The main concept of our project is the design and implementation of a fully functional vending machine controller using VHDL on the FPGA platform (Nexys A7-100T), along with the custom PCB prototype. The vending machine controller manages user interaction through inputs (push buttons, switches, reset) and displays information via outputs (7-segment display and LEDs).



As Figure 1 shows, the block diagram of the vending machine controller explains different components used in the project. First, the push button BTNC, 3 switches, reset and *inserted_amount* from *Coin_Handler* are used for state transition logic in *FSM_Controller* unit. Secondly, 4 push buttons (BTNU, BTNL, BTNR, BTND) and reset are used for coin inserting logic in the *Coin_Handler*. Lastly, reset and *display_text* are used for display logic in *Display_Manager* unit.

Also, from the block diagram, one can see that four LEDs are used as outputs from the *FSM_Controller* unit. Furthermore, *Display_Manager* logic will use a 7-segment decoder for displaying the output.



In Figure 2, the state machine diagram explains the core of the system in more detail. To be more specific, it is based on the Moore Finite State Machine that transitions through four different states: IDLE, DISPLAY, INSERT, DISPENSE. This FSM is responsible for controlling how the vending machine responds to the user actions:

- In IDLE, the system waits for BTNC_pressed to start interaction
- In DISPLAY, the user chooses an item using switches, and the system displays the selected product and price.
- In INSERT, the user inserts coins using push buttons, and system displays the required price and the inserted amount.
- In DISPENSE, if the inserted amount is greater than or equal to the required price, then it displays the balance and dispenses the product.

3.2 Requirements

Before we begin with the actual implementation, we formulated our requirements. These requirements evolved during the development period and the final requirements are as follows:

3.2.1 VHDL (Choi)

- Clearly defined Moore's Finite State Machine with four states (IDLE, DISPLAY, INSERT, DISPENSE)
- Encoding of user interactions
 - Push buttons for coin insertion logic (BTNU = 10 cents, BTNL = 20 cents, BTNR = 50 cents, and BTND = 100 cents = 1 euro)
 - BTNC for state transition, item selection confirmation, inserted amount confirmation
 - Switches for item selection
- Display item names, their prices, and inserted amounts on an 8-digits of 7-segment display
- Calculation of change and display the change
- Simulation of picking the dispensed item from the vending machine by switching off the selected item switch
- Ensuring the state transitions and outputs are synchronized to the clock signal
- Verification using test benches with the maximum coverage as possible

3.2.2 FPGA (*Leano*)

For successful integration of the vending machine logic into FPGA hardware, the following requirements must be met:

- Synthesis of VHDL FSM Design onto the FPGA using Vivado, ensuring compatibility with the chosen board.
- Pin constraints (XDC file) must be properly assigned to bind logical inputs/outputs to physical hardware (buttons, switches, 7-segment display).
- Clock signal synchronization must be maintained across all modules. The system should operate in sync with the 100 MHz onboard clock to ensure consistent state transitions and output behavior.
- Proper instantiation of 7-segment decoder modules to display selected items, prices, inserted amounts, and change across 8 digits.
- Efficient resource mapping during synthesis and implementation to stay within the available LUTs, flip-flops, and I/O pins.
- Bitstream (.bit) generation and FPGA programming must be verified using Vivado hardware manager.
- Final design must be tested on hardware to validate that all inputs (buttons, switches) and outputs (display, item logic) work correctly in real-time.

This FPGA implementation ensures real-time interaction with the vending machine interface and bridges the logical VHDL design with the physical hardware environment.

3.2.3 PCB (*Emaan*)

For creating great PCB design, we have to fulfil these requirements. [6]

- Make sure the PCB outline is closed and clean so software and machines can understand where board starts and ends
- Every component must have the correct footprint (pad size, shape and spacing) matching the part you place on board
- Set the rules how wide your tracks should be, how far they can be and the size of vias (also known as holes).
- Place parts logically. For example, put power components together, keep signals flowing in a clear direction, and leave space for wiring.
- Plan how the electrical lines (tracks) will be drawn. Keep high-speed or sensitive signals short and away from noisy signals. Match track lengths if needed.
- Avoid short circuits by ensuring enough space between pads and tracks.
- Use Design Rule Checker (DRC) to find missing connections, fix and find any spacing.
- Use KiCad 3D view to check how board will look in real life.

4. Explanation of Important Used Technologies

4.1 VHDL, ModelSim and Test Benches [3,4,5] (*Choi*)

VHDL stands for Very High-Speed Integrated Circuit Hardware Description Language. It means it is the hardware description language used to model and design electronic systems such as digital logic circuits. It allows description of the structure and behavior of hardware at various abstraction levels. In this project, VHDL was used to implement the core logic of the vending machine controller as Finite State Machine. This allowed us to have a precise control of the system's behavior across different states. VHDL also supports simulation and verification by using tools such as ModelSim.

ModelSim supports different HDL such as VHDL, Verilog, etc. This tool was specifically useful for us because we could not only write the code but also simulate the behavior of the code and analyze the waveforms of test benches for verification purpose.

Test bench is what we write to verify the functionalities of the implemented hardware design. It is simulating the device under test with our predefined input sequences to be tested and allows us to check for the expected outcome. This is a very important step before deploying our design to the FPGA board because it ensures the design works as we intend it to work.

4.2 FPGA and Vivado (*Leano*)

FPGA stands for **Field Programmable Gate Array**. It is a type of integrated circuit that can be **configured by the user after manufacturing**. Unlike traditional microcontrollers or processors, FPGAs contain **an array of logic blocks** and **reconfigurable interconnects**, allowing custom digital circuits to be built within the chip.

FPGAs enable parallel data processing and are widely used in:

- Real-time systems
- Signal processing
- Communication systems
- Embedded hardware acceleration

Vivado is a development environment provided by Xilinx (now part of AMD) for designing and programming FPGAs. It allows users to:

- Design digital logic using VHDL, Verilog, or graphical block design
- Simulate and verify the behavior of circuits
- Synthesize the logic into actual hardware implementation
- Generate bitstreams for programming the FPGA

Vivado supports a range of Xilinx FPGA devices and includes tools like logic analyzers, IP integrators, and constraint editors.

4.3 PCB and KiCad (*Emaan*)

PCB (Printed Circuit Board) is a flat board that holds and connects electronic components using wires. PCB prototyping is a process of designing and creating a test version of PCB before full scale production. It helps to check if circuit design works as expected and allows for fixing any errors early. [7] PCB prototyping is important because it can find design issues before production, saves money by avoiding reconstruction, it ensures the circuit performs as needed.

PCB has the following advantages and disadvantages

Pros of using PCB prototyping are:

Early detection of design flaws, quick feedback for design improvement, saves time.

Cons of using PCB prototyping are:

Can be time consuming for complex designs, it can be expensive to buy materials and fabrication, may require multiple iterations and limited by tools or fabrication resources.

I used KiCad (schematics, 3D viewer, routing, Gerber files and BOM), Online tool I used for PCB cost of manufacturing is PCBway. KiCad is a free, open-source PCB design tool that is widely used by students, hobbyists, and professionals for electronics design.

5. VHDL Implementation and Results (Choi)

5. 1 VHDL Implementation

The VHDL implementation of the vending machine controller was designed with modular and hierarchical structure. The system consists of top module and three sub components:

FSM_Controller, *Coin_Handler*, and *Display_Manager*.

The top module, *VendingMachine_Top* instantiates and connects all the subcomponents. What is also important to mention is that the overall system behavior is carried out by the Moore FSM. The difference between the Moore FSM and Mealy FSM is that the Moore FSM has its output decided only by the current state, unlike the Mealy FSM having its output decided by both the current state and the input. To be more specific, even though we have different possibilities of user inputs in each state, **the output of the FSM is the state indicator** (using the LEDs) and this is not dependent on the user input, such as button pressed for coin insertion. This proves that the FSM we implemented is Moore's FSM.

VendingMachine_Top ([See 9.1.1 VendingMachine_Top.vhd](#))

This is the top-level entity that connects the three sub-components and defines the interface with the physical inputs and outputs of the FPGA board.

1. We used the important libraries: IEEE.STD_LOGIC_1164 and IEEE.NUMERIC_STD. We used them because these are the most common and widely supported VHDL libraries and it also suits our purposes. To be more specific, we used IEEE.STD_LOGIC_1164 to declare all the basic signals (*std_logic* and *std_logic_vector*). These types represent digital signals which are essential for hardware. Also, the use of IEEE.STD_LOGIC_1164 library allowed us to use signed and unsigned type and allowed us to do arithmetic operations. The use of these libraries are universal to the submodules too.
2. Input ports and output ports are defined. The two key signals that are present in the code to be shared between different modules are *display_text_sig* and *inserted_amount_sig*. The signal, *display_text_sig*, drives the 8 digits display content and the signal, *inserted_amount_sig*, tracks the current balance of inserted coins.

FSM_Controller ([See 9.1.2 FSM_Controller.vhd](#))

This is the component that implements the core state machine logic of the vending machine along with other key functionalities.

1. First, Input ports and output ports are defined.
2. Then, two key functions, *string_to_ascii* and *format_price_to_string*, are defined.

First, the function, *string_to_ascii*, converts 8-characters string to a 64-bit *std_logic_vector*. We implemented this function because of the other component, *Display_Manager* expects a *std_logic_vector* (63 down to 0) to drive the 8-digit 7-segment display.

Secondly, the function, *format_price_to_string*, converts 14-bit unsigned integer(cents) to 4-digit ASCII character string. We implemented this function in order to convert the price and amount that was stored as unsigned (13 down to 0) into a decimal string on the 7-segment display.

3. Then, the signals as well as state types are declared to establish a linkage of usage across different components sharing the signals. These signals are *current_state*, *next_state*, *BTNC_prev*, *BTNC_edge*, *display_text_internal*, and *selected_price*.
4. "display_text <= display_text_internal" simply connects the internal display data to the display output.

5. The first process using only clock is intended to detect when the BTNC is pressed with rising edge detection.
6. The second process using clock and reset is the main state register of the FSM. This is the standard way to synchronize FSM states with the clock.
7. The next third process is the transition logic of the FSM. In the process, the transition to the next state, in other words, assignment of the correct next_state, happens based on the BTNC_edge, SW, inserted_amount, and selected_price. The logic of the FSM is already explained. The indicator for the current_state is LEDs. Depending on the current state, different LED will be lighted. The method we used is called one-hot encoding, which means that each FSM state activates exactly one LED where only one bit is 1. Also, from these lines of codes, we can say that the FSM is Moore FSM because the output of the FSM is the LED status, which is not affected by the user input.
8. Lastly, the last process is the display logic. This process controls the contents on the 8-digits 7-segment display, depending on the current_state, SW, inserted_amount, and the selected_price.

Coin_Handler (See 9.1.3 Display_Manager.vhd)

This is the component that detects the rising edge on the four coin buttons (BTNU, BTNL, BTNR and BTND) and increments the inserted_amount signal respectively.

Display_Manager (See 9.1.4 Coin_Handler.vhd)

This is the component that converts 64-bit display_text input (which represents 8 ASCII characters) into multiplexed outputs in order to make use of the 8-digits 7-segment display on the FPGA board.

5.2 Verification with Test benches

In order to ensure the correctness of the implemented design in VHDL, we need to perform simulation with test benches. Test benches were thus written for three components:

FSM_Controller.vhd, *Coin_Handler*, and *Display_Manager*. The test bench for the top module was not required.

FSM_Controller_tb

The test bench for FSM controller had 15 test cases.

- Test case 1: Check if the state transitions from IDLE to DISPLAY when the BTNC is pressed
- Test case 2: Check if the state transitions from DISPLAY to INSERT when no switch is selected
- Test case 3: Check if the state transitions from DISPLAY to INSERT when switch 1 is selected
- Test case 4: Check if the state transitions from INSERT to DISPENSE when inserted amount is lower than the price of the item
- Test case 5: Check if the state transitions from INSERT to DISPENSE when inserted amount is greater or equal to the price of the item
- Test case 6: Check if the state transitions from DISPENSE to IDLE when a switch is still on
- Test case 7: Check if the state transitions from DISPENSE to IDLE when a switch is finally off.
- Test case 8: Check if the state transitions from IDLE to DISPLAY when the BTNC is pressed.
- Test case 9: Check if the item 1 is displayed correctly if the SW(0) is on. (SW_tb = "001")
- Test case 10: Check if the item 2 is displayed correctly if the SW(1) is on. (SW_tb = "010")
- Test case 11: Check if the item 3 is displayed correctly if the SW(2) is on. (SW_tb = "100")

- Test case 12: Check if the inserted amount (50) is correctly being displayed on the right 4 digits of 7 segment display.
- Test case 13: Check if the inserted amount (150) allows state transition to DISPENSE and the correct change is displayed on the 7 segment display.
- Test case 14: Check if the state transitions from DISPENSE to IDLE when a switch is still on.
- Test case 15: Check if the state transitions from DISPENSE to IDLE when the switch is finally off.

And the following is the result.

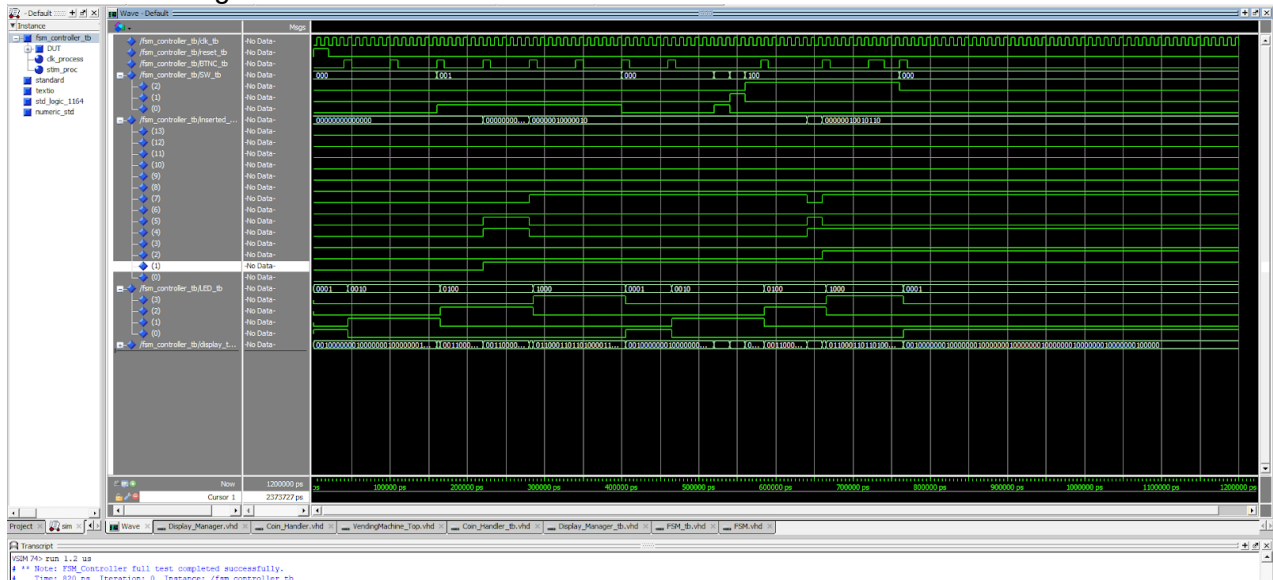


Figure 3. Results of FSM_Controller test bench [1]

Display Manager tb

The test bench for *Display_Manager* aimed to verify the correct 7-segment display patterns were generated for 6 different input strings. Those are:

- '01234567'
- 'ghlor '
- 'bEEr0150'
- '89 AEObc'
- 'colA0130'
- 'h2o 0090'

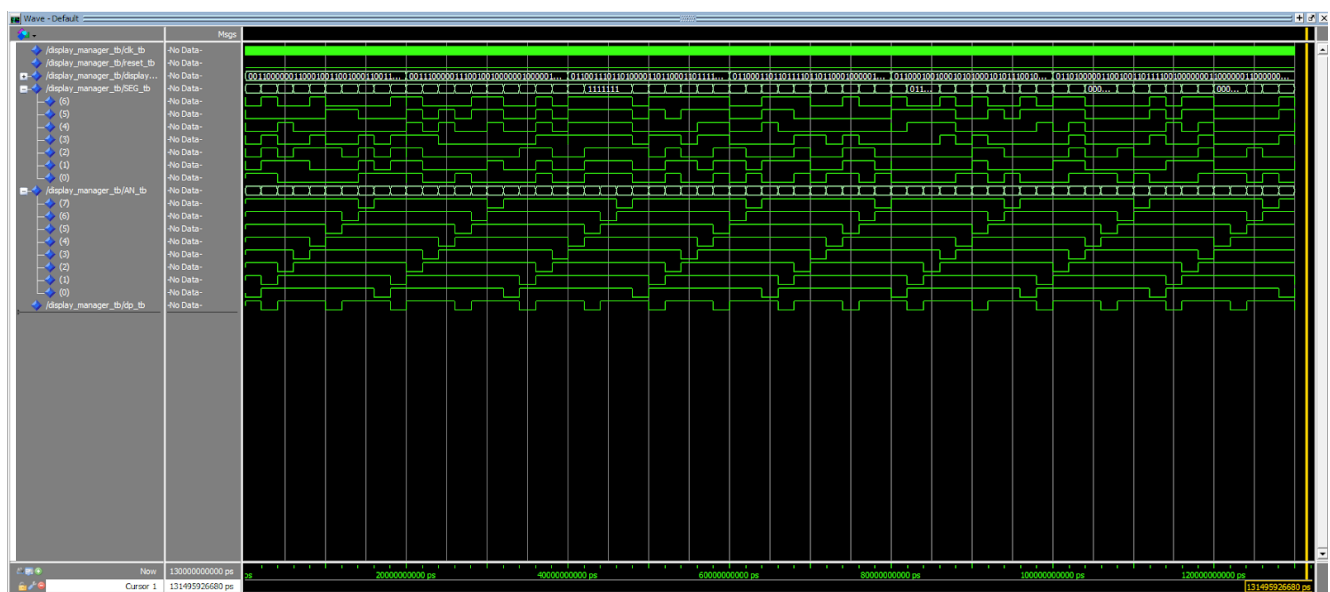


Figure 4. Results of Display_Manager test bench [1]

Coin_Handler_tb

The *Coin_Handler* test bench verified the component detects the rising edge of the buttons used for the coin insertion and increments the inserted amount. Those are:

- Insert 10 cents and see if the inserted amount is 10 cents
- Insert 20 cents and see if the inserted amount is 30 cents
- Insert 50 cents and see if the inserted amount is 80 cents
- Insert 100 cents = 1 euro and see if the inserted amount is 180 cents

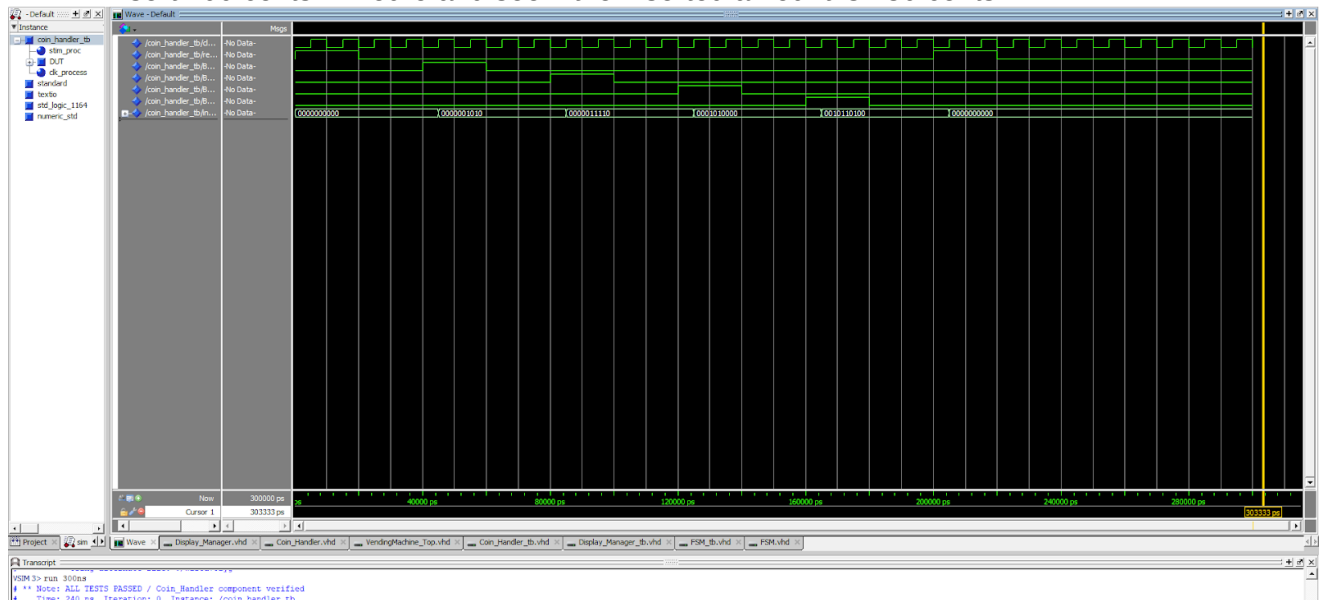


Figure 5. Results of Coin_Handler test bench [1]

6. FPGA Implementation and Results (*Leano*)

6.1 Overview of Project

This project implements one simple vending machine system based on FPGA. The user can select items, and insert different coins by using buttons. The system will detect the inserted value, and calculate the change, and interact with users by displaying output on 7-segment display. The FPGA used in our project is Diligent Nexys A7 and the supported software is Vivado. The block diagram of this project can be seen in the figure followed.

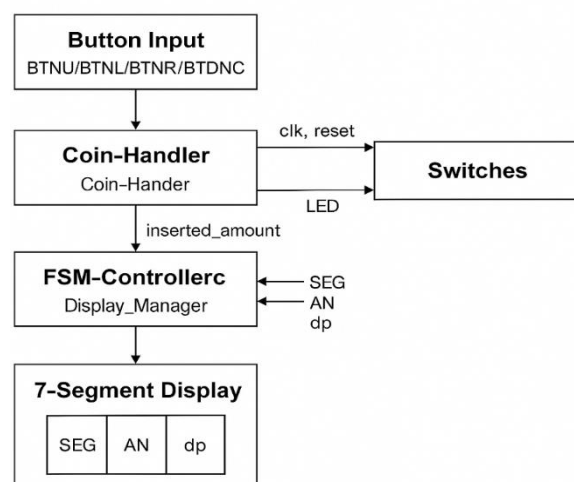


Figure 6. Block Diagram [1]

6.2 System Architecture

In the VHDL code, we have four modules: **Coin_Handler**, **FSM_Controller**, **Display_Manager**, and **Top Module**.

Coin_Handler

In this module, the system detects button presses and increases the inserted amount accordingly. Supports 4 coin values via BTNU, BTND, BTNL, BTNR (10, 100, 20, 50 cents respectively).

FSM_Controller

Main finite state machine controlling all vending machine logic: item selection, price validation, state transitions (IDLE, DISPLAY, INSERT, DISPENSE), LED status, and display string generation. Button BTNC is used to transition states, and switch M13, L16, and J15 are used to select different items in Display state.

Display_Manager

This module converts an 8-character ASCII string to 7-segment codes. Uses time-multiplexing to scan 8 digits and supports dynamic display updates.

Top Module : VendingMachine_Top

Instantiates and connects all modules: **Coin_Handler**, **FSM_Controller**, and **Display_Manager**.

In the xdc file below, the connection between VHDL module ports and FPGA board is provided, here you can observe the mapping between switches.

```

1  # Clock input (used for digit switching)
2  set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports { clk }];
3  create_clock -add -name sys_clk_pin -period 20.00 -waveform {0 10} [get_ports {clk}];
4  # reset
5  set_property PACKAGE_PIN V10 [get_ports {reset}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {reset}]
7  # buttons
8  set_property PACKAGE_PIN N17 [get_ports {BTNC}]
9  set_property IOSTANDARD LVCMOS33 [get_ports {BTNC}]
10 set_property PACKAGE_PIN M18 [get_ports {BTNU}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {BTNU}]
12 set_property PACKAGE_PIN P17 [get_ports {BTNL}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {BTNL}]
14 set_property PACKAGE_PIN M17 [get_ports {BTNR}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {BTNR}]
16 set_property PACKAGE_PIN P18 [get_ports {BTND}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {BTND}]
18 # sw
19 set_property PACKAGE_PIN J15 [get_ports {SW[0]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {SW[0]}]
21 set_property PACKAGE_PIN L16 [get_ports {SW[1]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {SW[1]}]
23 set_property PACKAGE_PIN M13 [get_ports {SW[2]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {SW[2]}]
25 # LED
26 set_property PACKAGE_PIN V11 [get_ports {LED[0]}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}]
28 set_property PACKAGE_PIN V12 [get_ports {LED[1]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {LED[1]}]
30 set_property PACKAGE_PIN V14 [get_ports {LED[2]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {LED[2]}]
32 set_property PACKAGE_PIN V15 [get_ports {LED[3]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {LED[3]}]
34 # 7 SEG[6:0]
35 # 7 段数码管段选 SEG[6:0]
36 set_property PACKAGE_PIN L18 [get_ports {SEG[0]}]
37 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[0]}]
38 set_property PACKAGE_PIN T11 [get_ports {SEG[1]}]
39 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[1]}]
40 set_property PACKAGE_PIN P15 [get_ports {SEG[2]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[2]}]
42 set_property PACKAGE_PIN K13 [get_ports {SEG[3]}]
43 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[3]}]
44 set_property PACKAGE_PIN K16 [get_ports {SEG[4]}]
45 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[4]}]
46 set_property PACKAGE_PIN R10 [get_ports {SEG[5]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[5]}]
48 set_property PACKAGE_PIN T10 [get_ports {SEG[6]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[6]}]
50 # Anode control
51 set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVCMOS33 } [get_ports { AN[7] }];
52 set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports { AN[6] }];
53 set_property -dict { PACKAGE_PIN T9 IOSTANDARD LVCMOS33 } [get_ports { AN[5] }];
54 set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports { AN[4] }];
55 set_property -dict { PACKAGE_PIN P14 IOSTANDARD LVCMOS33 } [get_ports { AN[3] }];
56 set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports { AN[2] }];
57 set_property -dict { PACKAGE_PIN K2 IOSTANDARD LVCMOS33 } [get_ports { AN[1] }];
58 set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports { AN[0] }];
59 # dp
60 set_property PACKAGE_PIN H15 [get_ports {dp}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {dp}]

```

Figure 7. Constraint File [1]

6.3 FPGA Synthesis & Implementation Results

Synthesis

The design was synthesized using Xilinx Vivado 2024.2 for the xc7a100tcs324-1 FPGA device. Below is the resource utilization summary:

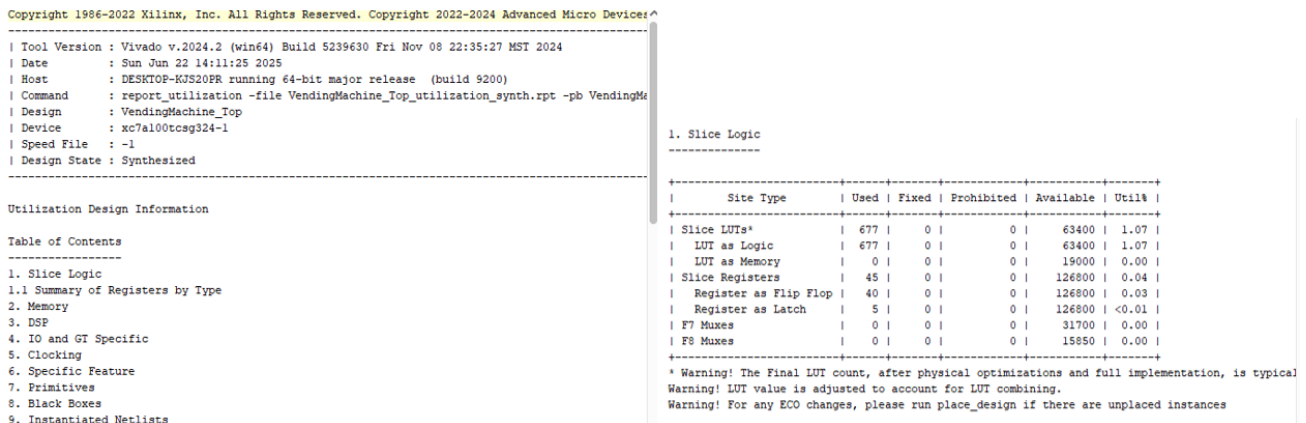


Figure 8. Synthesis Report [1]

Hardware Implementation

After synthesis:

- Bitstream generated and loaded successfully onto the FPGA board.
- Timing met for target frequency.
- The device is programmed.

7. PCB Prototyping and Results (Emaan)

To implement the PCB design, I started by creating the circuit schematic, selecting and connecting all necessary components. Then, I assigned footprints, arranged the layout, and routed the connections using KiCad. Finally, I reviewed the 3D view and design rules to ensure everything was ready for fabrication.

The complete PCB design provides a custom hardware platform for vending machine FPGA. It includes all essential components and allows for future expansion using extra I/O pins. Below are the final stages of our PCB design:

7.1 Schematic Design

The schematic is the first step in PCB development. It shows the logical circuit, including all components and their connections, ensuring correct circuit behavior before layout. Our schematic includes, as can be seen in Figure 9:

- The main FPGA chip
- Power supply circuit
- 7-segment display connections
- Resistors and Capacitors
- RGB LED circuits
- Transistors
- Switch connections

7.3 Routing

After completing the schematic, the next step was to translate the logical connections into physical traces on the PCB. The routing process is crucial for ensuring that the final PCB will function correctly and be manufacturable. The routing process involved:

- Placing components in optimal positions for signal integrity and ease of use
- Routing traces between components, ensuring proper clearances and adhering to design rules
- Creating separate power and ground planes for better signal integrity
- Minimizing trace lengths for critical signals

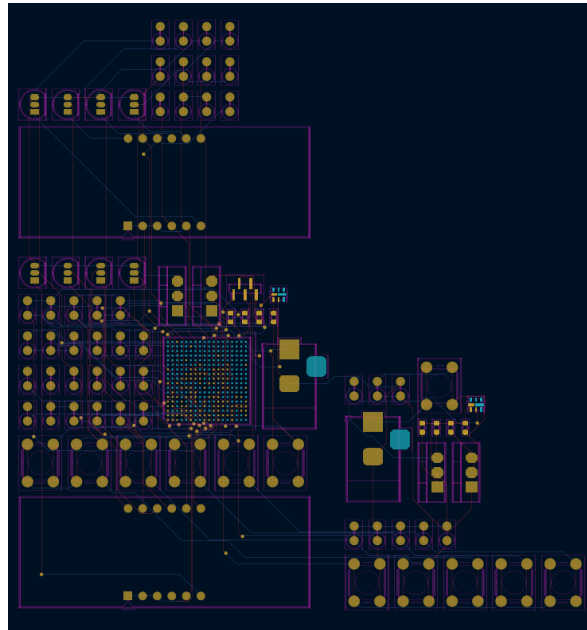


Figure 11. PCB Layout [1]

7.4 BOM and Cost for manufacturing Using Gerber Files

According to the BOM list for this PCB we did use total 12 different elements. For Generating Building Of Material (BOM) I used a built in KiCad plugin, generate Interactive HTML BOM, to find out the cost for manufacturing I upload the Gerber file on JLCPCB.

	Sour ced	Plac ed	References	Value	Footprint	Quantity
1	☐	☐	C1, C2, C3, C4, C5, C6, C7, C8	C	C_0603_1608Metric	8
2	☐	☐	R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31, R32, R33, R34, R35, R36, R37, R38, R39, R40, R41, R42, R43	R	R_Axial_DIN0207_L6.3mm_D2.5mm_P2.54mm_V ertical	43
3	☐	☐	D1, D2	LED	LED-APA102-2020	2
4	☐	☐	U1, U5	LM7805_T0220	TO-220-3_Vertical	2
5	☐	☐	U2, U6	LM317_T0-220	TO-220-3_Vertical	2
6	☐	☐	U3, U7	CA56-12EWA	CA56-12EWA	2
7	☐	☐	U4	XC7A100T-CSG324	BGA-324_15x15mm_Layout18x18_P0.8mm	1
8	☐	☐	SW2, SW3, SW4, SW5, SW6, SW8, SW9, SW10, SW11, SW12	Sw_Push	SW_PUSH_6mm_H13mm	10
9	☐	☐	SW1, SW7	SW_SPST	SW_PUSH_6mm_H13mm	2
10	☐	☐	Q1, Q2, Q3, Q4, Q5, Q6, Q7, Q8	BC557	TO-92_Inline	8
11	☐	☐	J1, J3	Barrel_Jack	BarrelJack_Horizontal	2
12	☐	☐	J2	Conn_01x05_Pin	PinSocket_1x05_P1.00mm_Vertical_SMD_Pin 1Left	1

Figure 12. Bill of Materials [1]

Charge Details

Engineering fee	\$31.00
Via Covering	\$0.00
Surface Finish	\$5.10
Board	\$7.10

Build Time

PCB:

- ☒ 5-6 days \$0.00
- ☐ 4-5 days \$62.90
- ☐ 3-4 days \$125.80
- ☐ 2-3 days \$188.60

Calculated Price

Additional charges may apply for special cases

\$43.20

SAVE TO CART

Shipping Estimate

\$22.73

▼ DHL Express (DDP) 2-4 business days

Weight 0.34kg

Coupons

Save \$30.00 Save \$20.00

Figure 13. Cost [1]

7.5 Gerber files

Gerber files are special files used to describe each layer of a PCB design. They contain instructions for drawing things like copper traces, holes (vias), pads, solder masks, and labels. These files are created by PCB design software and are used by manufacturers to build the actual circuit board. These files often have extensions like `.gbr` or `.gbr`. [8]

☒	1 vending_machine_pcb-B_Cu.gbr (Copper, L2)
☒	2 vending_machine_pcb-B_Paste.gbr (Paste, Bot)
☒	3 vending_machine_pcb-Edge_Cuts.gbr (Profile, NP)
☒	4 vending_machine_pcb-F_Mask.gbr (Soldermask, Top)
☒	5 vending_machine_pcb-F_Silkscreen.gbr (Legend, Top)
☒	6 vending_machine_pcb-B_Mask.gbr (Soldermask, Bot)
☒	7 vending_machine_pcb-B_Silkscreen.gbr (Legend, Bot)
☒	8 vending_machine_pcb-F_Cu.gbr (Copper, L1)
☒	9 vending_machine_pcb-F_Paste.gbr (Paste, Top)

Figure 14. Gerber Files Layers [1]

Board stats	Front	Back	Total
Components	83	0	83
Groups	12	0	12
SMD pads	361	0	361
TH pads	200		
Checkboxes			
Sourced	0/83 (0%)		
Placed	0/83 (0%)		

Figure 15. Board Stats [1]

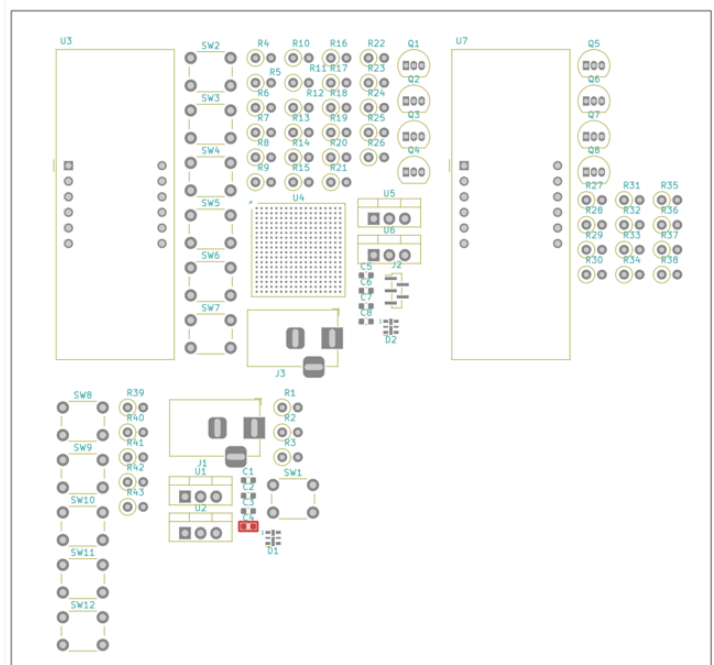


Figure 16. PCB Image [1]

8. Sources and References (*Everyone*)

[1] https://github.com/yoonsukwkd96/SS2025_HWE_Lab_Team1_VendingMachine

[2] <https://agilealliance.org/agile101/>

[3] IEEE Standard VHDL Language Reference Manual, IEEE 1076-2019.

[4] Siemens EDA - ModelSim Product Overview <https://eda.sw.siemens.com/en-US/ic/model/>

[5] Roth, C. H., & John, L. L. (2016). Digital Systems Design Using VHDL, 3rd Edition.

[6] Cadence PCB Solutions (2023) PCB design layout guidelines and Best Practices for Engineers, Cadence.

[7] Peterson , Z. (2020) What is a printed circuit board? make circuits by connecting components, Altium.

[8] Team, V.E. (2024) Gerber Files explained, VSE.

[9] Final proof reading and compilation of the report and repository organization were done by Younsuk Choi

9. Appendix (Everyone)

Every single material on this report can be found on our project Git Repository. [1]

9.1 VHDL Codes

9.1.1 VendingMachine_Top.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity VendingMachine_Top is
6  Port (
7      clk      : in  std_logic;
8      reset    : in  std_logic;
9      BTNC     : in  std_logic;
10     BTNU     : in  std_logic;
11     BTNL     : in  std_logic;
12     BTNR     : in  std_logic;
13     BTND     : in  std_logic;
14     SW       : in  std_logic_vector(2 downto 0);
15     LED      : out std_logic_vector(3 downto 0);
16     SEG      : out std_logic_vector(6 downto 0);
17     AN       : out std_logic_vector(7 downto 0);
18     dp       : out std_logic;
19 );
20 end VendingMachine_Top;
21
22 architecture Behavioral of VendingMachine_Top is
23     signal display_text_sig : std_logic_vector(63 downto 0);
24     signal inserted_amount_sig : unsigned(13 downto 0);
25 
```

```

26 begin
27     fsm_inst : entity work.FSM_Controller
28     port map (
29         clk      => clk,
30         reset    => reset,
31         BTNC     => BTNC,
32         SW       => SW,
33         inserted_amount => inserted_amount_sig,
34         LED      => LED,
35         display_text => display_text_sig
36     );
37
38     coin_inst : entity work.Coin_Handler
39     port map (
40         clk      => clk,
41         reset    => reset,
42         BTNU     => BTNU,
43         BTNL     => BTNL,
44         BTNR     => BTNR,
45         BTND     => BTND,
46         inserted_amount => inserted_amount_sig(9 downto 0)
47     );
48
49     disp_inst : entity work.Display_Manager
50     port map (
51         clk      => clk,
52         reset    => reset,
53         display_text => display_text_sig,
54         SEG      => SEG,
55         AN       => AN,
56         dp       => dp
57     );
58
59 end Behavioral;
60 
```

9.1.2 FSM_Controller.vhd

```

5  entity FSM_Controller is
6      Port (
7          clk          : in  std_logic;
8          reset        : in  std_logic;
9          BTNC         : in  std_logic;
10         SW           : in  std_logic_vector(2 downto 0);
11         inserted_amount : in  unsigned(13 downto 0);
12         LED           : out std_logic_vector(3 downto 0);
13         display_text  : out std_logic_vector(63 downto 0)
14     );
15 end FSM_Controller;
16
17 architecture Behavioral of FSM_Controller is
18
19     function string_to_ascii(s : string) return std_logic_vector is
20         variable result : std_logic_vector(63 downto 0) := (others => '0');
21     begin
22         for i in 0 to 7 loop
23             if i < s'length then
24                 result(63 - i*8 downto 56 - i*8) := std_logic_vector(to_unsigned(character'pos(s(i+1)), 8));
25             else
26                 result(63 - i*8 downto 56 - i*8) := x"20"; -- space
27             end if;
28         end loop;
29         return result;
30     end function;
31
32     function format_price_to_string(value : unsigned(13 downto 0)) return string is
33         variable str : string(1 to 4);
34         variable int_val : integer := to_integer(value);
35     begin
36         str(1) := character'val(int_val / 1000 mod 10 + character'pos('0'));
37         str(2) := character'val(int_val / 100 mod 10 + character'pos('0'));
38         str(3) := character'val(int_val / 10 mod 10 + character'pos('0'));
39         str(4) := character'val(int_val mod 10 + character'pos('0'));
40         return str;
41     end function;
42
43     type state_type is (IDLE, DISPLAY, INSERT, DISPENSE);
44     signal current_state, next_state : state_type;
45     signal BTNC_prev : std_logic := '0';
46     signal BTNC_edge : std_logic;
47     signal display_text_internal : std_logic_vector(63 downto 0);
48     signal selected_price : unsigned(13 downto 0) := (others => '0');
49
50 begin
51
52     display_text <= display_text_internal;
53
54     -- Detect rising edge of BTNC
55     process(clk)
56     begin
57         if rising_edge(clk) then
58             BTNC_prev <= BTNC;
59         end if;
60     end process;
61
62     BTNC_edge <= '1' when (BTNC = '1' and BTNC_prev = '0') else '0';
63
64     -- State register
65     process(clk, reset)
66     begin
67         if reset = '1' then
68             current_state <= IDLE;
69         elsif rising_edge(clk) then
70             current_state <= next_state;
71         end if;
72     end process;
73

```

```

74 -- FSM transitions
75 process(current_state, BTNC_edge, SW, inserted_amount, selected_price)
76 begin
77     case current_state is
78     when IDLE =>
79         if BTNC_edge = '1' then
80             next_state <= DISPLAY;
81         else
82             next_state <= IDLE;
83         end if;
84
85     when DISPLAY =>
86         if BTNC_edge = '1' and (SW = "001" or SW = "010" or SW = "100") then
87             next_state <= INSERT;
88         else
89             next_state <= DISPLAY;
90         end if;
91
92     when INSERT =>
93         if BTNC_edge = '1' and inserted_amount >= selected_price then
94             next_state <= DISPENSE;
95         else
96             next_state <= INSERT;
97         end if;
98
99     when DISPENSE =>
100         if BTNC_edge = '1' and SW = "000" then
101             next_state <= IDLE;
102         else
103             next_state <= DISPENSE;
104         end if;
105
106     when others =>
107         next_state <= IDLE;
108     end case;
109 end process;
110
111 -- LED output
112 with current_state select
113     LED <= "0001" when IDLE,
114           "0010" when DISPLAY,
115           "0100" when INSERT,
116           "1000" when DISPENSE;
117
118 -- Display logic
119 process(current_state, SW, inserted_amount, selected_price)
120 variable price_str : string(1 to 4);
121 variable inserted_str : string(1 to 4);
122 variable full_str : string(1 to 8);
123 variable temp_change : unsigned(13 downto 0);
124 variable chg_str : string(1 to 4);
125 begin
126     case current_state is
127     when IDLE =>
128         if SW = "001" then
129             selected_price <= to_unsigned(130, 14);
130             display_text_internal <= string_to_ascii("01300000");
131         elsif SW = "010" then
132             selected_price <= to_unsigned(150, 14);
133             display_text_internal <= string_to_ascii("01500000");
134         elsif SW = "100" then
135             selected_price <= to_unsigned(90, 14);
136             display_text_internal <= string_to_ascii("00900000");
137         else
138             selected_price <= (others => '0');
139             display_text_internal <= string_to_ascii(" ");
140         end if;
141     when DISPLAY =>
142         case SW is
143         when "001" =>
144             selected_price <= to_unsigned(130, 14);
145             display_text_internal <= string_to_ascii("c01A0130");
146         when "010" =>
147             selected_price <= to_unsigned(150, 14);
148             display_text_internal <= string_to_ascii("bEE0150");
149         when "100" =>
150             selected_price <= to_unsigned(90, 14);
151             display_text_internal <= string_to_ascii("h20 0090");
152         when others =>
153             display_text_internal <= string_to_ascii(" ");
154         end case;
155     end case;
156
157     when INSERT =>
158         price_str := format_price_to_string(selected_price);
159         inserted_str := format_price_to_string(inserted_amount);
160         full_str := price_str & inserted_str;
161         display_text_internal <= string_to_ascii(full_str);
162
163     when DISPENSE =>
164         if inserted_amount >= selected_price then
165             temp_change := inserted_amount - selected_price;
166         else
167             temp_change := (others => '0');
168         end if;
169
170         chg_str(1) := character'val(to_integer(temp_change / 1000 mod 10) + character'pos('0'));
171         chg_str(2) := character'val(to_integer(temp_change / 100 mod 10) + character'pos('0'));
172         chg_str(3) := character'val(to_integer(temp_change / 10 mod 10) + character'pos('0'));
173         chg_str(4) := character'val(to_integer(temp_change mod 10) + character'pos('0'));
174
175         display_text_internal <= string_to_ascii("chg" & chg_str);
176
177     when others =>
178         display_text_internal <= string_to_ascii(" ");
179     end case;
180 end process;
181 end Behavioral;

```

9.1.3 Display_Manager.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Display_Manager is
6  Port (
7      clk          : in  std_logic;
8      reset        : in  std_logic;
9      display_text : in  std_logic_vector(63 downto 0);
10     SEG          : out std_logic_vector(6 downto 0);
11     AN           : out std_logic_vector(7 downto 0);
12     dp           : out std_logic
13 );
14 end Display_Manager;
15
16 architecture Behavioral of Display_Manager is
17
18     signal clk_divider : unsigned(17 downto 0) := (others => '0');
19     signal digit_index : integer range 0 to 7 := 0;
20     signal current_char : std_logic_vector(7 downto 0);
21     signal seg_temp : std_logic_vector(6 downto 0);
22
23 begin
24     process(clk)
25     begin
26         if rising_edge(clk) then
27             if reset = '1' then
28                 clk_divider <= (others => '0');
29                 digit_index <= 0;
30             else
31                 if clk_divider = 200000 - 1 then
32                     clk_divider <= (others => '0');
33                     if digit_index = 7 then
34                         digit_index <= 0;
35                     else
36                         digit_index <= digit_index + 1;
37                     end if;
38                 else
39                     clk_divider <= clk_divider + 1;
40                 end if;
41             end if;
42         end if;
43     end process;
44
45     current_char <= display_text(63 - digit_index*8 downto 56 - digit_index*8);
46
47     process(current_char)
48     begin
49         case current_char is
50             when x"20" => seg_temp <= "1111111"; -- space
51             when x"30" => seg_temp <= "0000001"; -- '0'
52             when x"31" => seg_temp <= "1001111"; -- '1'
53             when x"32" => seg_temp <= "0010010"; -- '2'
54             when x"33" => seg_temp <= "0000110"; -- '3'
55             when x"34" => seg_temp <= "1001100"; -- '4'
56             when x"35" => seg_temp <= "0100100"; -- '5'
57             when x"36" => seg_temp <= "0100000"; -- '6'
58             when x"37" => seg_temp <= "0001111"; -- '7'
59             when x"38" => seg_temp <= "0000000"; -- '8'
60             when x"39" => seg_temp <= "0000100"; -- '9'
61             when x"41" => seg_temp <= "0001000"; -- 'A\'
62             when x"45" => seg_temp <= "0110000"; -- 'E'
63             when x"4F" => seg_temp <= "0000001"; -- 'O'
64             when x"62" => seg_temp <= "1100000"; -- 'b'
65             when x"63" => seg_temp <= "1110010"; -- 'c'
66             when x"67" => seg_temp <= "0000100"; -- 'g'
67             when x"68" => seg_temp <= "1101000"; -- 'h'
68             when x"6C" => seg_temp <= "1001111"; -- 'l'
69             when x"6F" => seg_temp <= "1100010"; -- 'o'
70             when x"72" => seg_temp <= "1111000"; -- 'r'
71             when others => seg_temp <= "1111111"; -- blank
72         end case;
73     end process;
74
75     SEG <= seg_temp;
76     AN <= not std_logic_vector(to_unsigned(2**digit_index, 8));
77     dp <= '0' when (digit_index = 1 or digit_index = 5) else '1';
78
79 end Behavioral;

```

9.1.4 Coin_Handler.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Coin_Handler is
6  Port (
7      clk          : in  std_logic;
8      reset        : in  std_logic;
9      BTNU, BTNL, BTNR, BTND : in std_logic;
10     inserted_amount : out unsigned(9 downto 0)
11 );
12 end Coin_Handler;
13
14 architecture Behavioral of Coin_Handler is
15
16     signal amount : unsigned(9 downto 0) := (others => '0');
17     signal prev_BTNU, prev_BTNL, prev_BTNR, prev_BTND : std_logic := '0';
18
19 begin
20
21     process(clk)
22     begin
23         if rising_edge(clk) then
24             if reset = '1' then
25                 amount <= (others => '0');
26             else
27                 if (BTNU = '1' and prev_BTNU = '0') then
28                     amount <= amount + 10;
29                 elsif (BTNL = '1' and prev_BTNL = '0') then
30                     amount <= amount + 20;
31                 elsif (BTNR = '1' and prev_BTNR = '0') then
32                     amount <= amount + 50;
33                 elsif (BTND = '1' and prev_BTND = '0') then
34                     amount <= amount + 100;
35                 end if;
36             end if;
37
38             prev_BTNU <= BTNU;
39             prev_BTNL <= BTNL;
40             prev_BTNR <= BTNR;
41             prev_BTND <= BTND;
42         end if;
43     end process;
44
45     inserted_amount <= amount;
46
47 end Behavioral;
48
```